

সম্পূর্ণ প্রকল্প ব্যাখ্যা

EmotionSense

টেক্সট থেকে আবেগ শনাক্তকরণ — একটি ফুল-স্ট্যাক মেশিন লার্নিং প্রকল্প



বিষয়: Natural Language Processing · Text Classification

মডেল: DistilBERT (Transformer) · ৬-শ্রেণির আবেগ শনাক্তকরণ

ডেটাসেট: dair-ai/emotion — ২০,০০০ ইংরেজি বাক্য

টেক-স্ট্যাক: PyTorch · FastAPI · Next.js 14 · Docker

লাইভ অ্যাপ: emotionsense-ochre.vercel.app

লাইভ API: emotionsense-api-452m.onrender.com

সোর্স কোড: github.com/claudeproject751-dot/tricore

সূচিপত্র

১. এক নজরে প্রকল্প — কী বানানো হয়েছে এবং কেন
২. অত্যন্ত জরুরি সতর্কতা — সুপারভাইজারকে যা অবশ্যই বলতে হবে
৩. মেশিন লার্নিং-এর মৌলিক ধারণা — একদম শূন্য থেকে
৪. ডেটাসেট — dair-ai/emotion বিস্তারিত
৫. টেক্সটকে সংখ্যায় রূপান্তর — Tokenization ও Embedding
৬. Transformer, BERT ও DistilBERT — মডেলের গঠন
৭. বেসলাইন মডেল — TF-IDF + Logistic Regression
৮. মূল ট্রেনিং: Fine-tuning — ধাপে ধাপে সম্পূর্ণ ব্যাখ্যা
৯. হাইপারপ্যারামিটার — প্রতিটির অর্থ ও যুক্তি
১০. মূল্যায়ন — Accuracy, Precision, Recall, F1
১১. ফলাফল বিশ্লেষণ — আসল সংখ্যা ও Confusion Matrix
১২. ব্যাকএন্ড — FastAPI ইনফারেন্স সার্ভিস
১৩. ফ্রন্টএন্ড — Next.js ইউজার ইন্টারফেস
১৪. ডিপ্লয়মেন্ট — Vercel, Render, Docker
১৫. টেস্টিং ও CI
১৬. সীমাবদ্ধতা ও নৈতিক দিক
১৭. সুপারভাইজারের সম্ভাব্য প্রশ্ন ও উত্তর — ৪০+ প্রশ্ন
১৮. পরিভাষা — ইংরেজি-বাংলা শব্দকোষ

১. এক নজরে প্রকল্প

১.১ সমস্যাটি কী?

মানুষ যখন কিছু লেখে, সেই লেখার ভেতরে একটা **আবেগ** লুকিয়ে থাকে। "আজ আমার জন্মদিন, খুব ভালো লাগছে" — এটা আনন্দের। "সবাই চলে গেছে, একা লাগছে" — এটা দুঃখের। মানুষ সহজেই বুঝতে পারে, কিন্তু কম্পিউটারকে বোঝানো কঠিন, কারণ কম্পিউটার শব্দের অর্থ জানে না — সে শুধু সংখ্যা বোঝে।

এই প্রকল্পের লক্ষ্য: একটি ইংরেজি বাক্য দিলে কম্পিউটার বলে দেবে সেটি ছয়টি আবেগের মধ্যে কোনটি প্রকাশ করছে — দুঃখ (sadness), আনন্দ (joy), ভালোবাসা (love), রাগ (anger), ভয় (fear), বিস্ময় (surprise)।

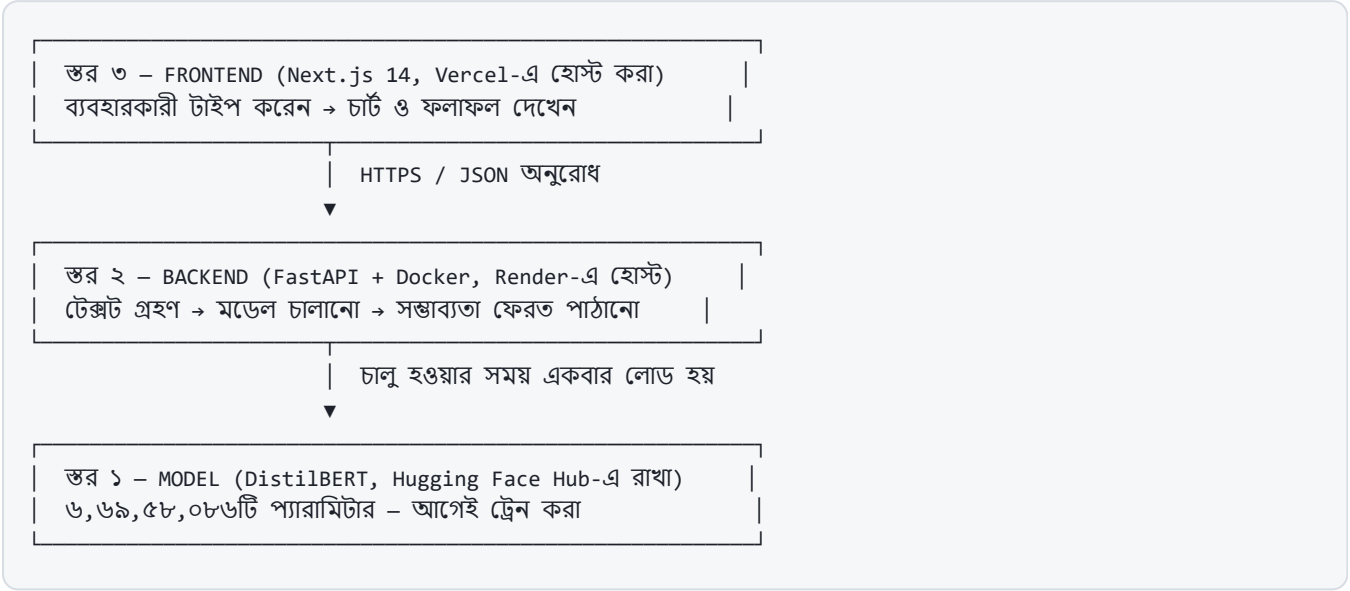
১.২ শুধু উত্তর নয়, আত্মবিশ্বাসও

সাধারণ শ্রেণিবিন্যাসকারী শুধু একটি উত্তর দেয় — "এটা আনন্দ"। কিন্তু এতে একটা বড় সমস্যা লুকিয়ে থাকে। মডেল যদি ৯৯% নিশ্চিত হয়ে "আনন্দ" বলে, আর ৩৫% নিশ্চিত হয়েও "আনন্দ" বলে — দুটোর মান এক নয়।

কিন্তু বাইরে থেকে দুটোই একই রকম দেখায়।

তাই এই সিস্টেম **ছয়টি শ্রেণির প্রত্যেকটির সম্ভাবনা** ফেরত দেয়, শুধু বিজয়ীটি নয়। ব্যবহারকারী দেখতে পান দ্বিতীয় স্থানে কে আছে এবং ব্যবধান কতটুকু। এটি একটি সচেতন ডিজাইন সিদ্ধান্ত — **অনিশ্চয়তা লুকানো নয়, দেখানো।**

১.৩ তিনটি স্তর



মনে রাখার মতো মূল কথা

ট্রেনিং হয় **একবার, আলাদাভাবে (offline)** — GPU-তে। ট্রেনিং শেষে মডেলটি একটি ফাইল হিসেবে সংরক্ষিত হয়। এরপর ব্যাকএন্ড শুধু সেই ফাইলটি লোড করে ব্যবহার করে। **ব্যবহারকারী যখন টাইপ করেন তখন কোনো ট্রেনিং হয় না** — শুধু ইনফারেন্স (inference) বা ভবিষ্যদ্বাণী হয়। এই পার্থক্যটা সুপারভাইজাররা প্রায়ই যাচাই করেন।

২. অত্যন্ত জরুরি সতর্কতা — এটি আগে পড়ুন

⚠ বর্তমানে যে মডেলটি চলছে সেটি আপনার নিজের ট্রেন করা নয়

এই মুহূর্তে লাইভ API যে মডেলটি ব্যবহার করছে তার নাম `bhadresh-savani/distilbert-base-uncased-emotion`। এটি Hugging Face-এ থাকা একটি পাবলিক মডেল, যা একই ডেটাসেট ও একই ছয়টি শ্রেণিতে অন্য কেউ ট্রেন করেছেন।

আপনার নিজস্ব ট্রেনিং এখনো চালানো হয়নি। ট্রেনিং-এর সম্পূর্ণ কোড ও নোটবুক তৈরি এবং প্রস্তুত আছে (`ml/notebooks/emotionsense_train_kaggle.ipynb`), শুধু Kaggle-এ চালানো বাকি — যা প্রায় ৬ মিনিটের কাজ।

সুপারভাইজারকে এটি স্পষ্টভাবে বলুন। "আমি DistilBERT ট্রেন করেছি" বলা এই মুহূর্তে সত্য নয় এবং এটি ধরা পড়লে পুরো কাজের বিশ্বাসযোগ্যতা নষ্ট হবে। সঠিক বাক্য হলো: "সম্পূর্ণ ট্রেনিং পাইপলাইন আমি তৈরি করেছি এবং বেসলাইন মডেল নিজে ট্রেন করেছি; ট্রান্সফরমার ফাইন-টিউনিং-এর জন্য আপাতত একই ডেটাসেটে ট্রেন করা একটি পাবলিক মডেল ব্যবহার করছি, নিজের ফাইন-টিউন চালানো বাকি।"

২.১ কোনটা আপনি নিজে করেছেন, কোনটা করেননি

অংশ	অবস্থা	প্রমাণ
ডেটা বিশ্লেষণ ও শ্রেণি-বন্টন	✅ নিজে চালানো	<code>class_distribution.json</code>
বেসলাইন মডেল ট্রেনিং (TF-IDF)	✅ নিজে ট্রেন করা	<code>baseline_metrics.json</code> — ৮৭.১৫%
ট্রান্সফরমার ট্রেনিং কোড	✅ নিজে লেখা	<code>train_transformer.py</code> , Kaggle নোটবুক
ট্রান্সফরমার আসলে ট্রেন করা	❌ এখনো চালানো হয়নি	—
ডিপ্লয় করা মডেলের মূল্যায়ন	✅ নিজে চালানো	<code>metrics.json</code> — ৯২.৭%
ব্যাকএন্ড API	✅ নিজে তৈরি	৪০টি টেস্ট পাস
ফ্রন্টএন্ড	✅ নিজে তৈরি	১৮টি টেস্ট পাস
ডিপ্লয়মেন্ট	✅ নিজে করা	লাইভ URL

ভালো খবর: ৯২.৭% যে সংখ্যাটি অ্যাপে দেখাচ্ছে, সেটি আপনি নিজে পরিমাপ করেছেন — টেস্ট সেটে ওই মডেলটি চালিয়ে। সংখ্যাটি কোথাও থেকে কপি করা নয়। এটাও একটি বৈধ ও মূল্যবান কাজ, যাকে বলে মডেল মূল্যায়ন (model evaluation)।

৩. মেশিন লার্নিং-এর মৌলিক ধারণা

৩.১ প্রোগ্রামিং বনাম মেশিন লার্নিং

সাধারণ প্রোগ্রামিং-এ আমরা **নিয়ম** লিখি: "যদি বাক্যে 'happy' শব্দ থাকে, তবে আনন্দ"। কিন্তু ভাষা এত সরল নয় — "I am not happy at all" বাক্যে 'happy' আছে, তবু এটি আনন্দ নয়। হাজার হাজার নিয়ম লিখেও ভাষা ঢাকা যায় না।

মেশিন লার্নিং উল্টো পথে হাঁটে। আমরা নিয়ম লিখি না — বরং **বহু উদাহরণ** দেখাই, আর যন্ত্র নিজে নিয়মগুলো খুঁজে বের করে।

সাধারণ প্রোগ্রামিং: নিয়ম + ডেটা → উত্তর
মেশিন লার্নিং: ডেটা + উত্তর → নিয়ম (মডেল)

আমাদের ক্ষেত্রে: ১৬,০০০টি বাক্য এবং প্রত্যেকটির সঠিক আবেগ যন্ত্রকে দেখানো হয়। যন্ত্র ধীরে ধীরে নিজেই ধরন (pattern) শিখে নেয়।

৩.২ সুপারভাইজড লার্নিং

আমাদের প্রতিটি উদাহরণের সাথে **সঠিক উত্তর** দেওয়া আছে। যেমন:

ইনপুট (x) — বাক্য	লেবেল (y) — সঠিক উত্তর
i didnt feel humiliated	০ = sadness
im grabbing a minute to post i feel greedy wrong	৩ = anger
i am feeling grouchy	৩ = anger

সঠিক উত্তর সহ শেখানোকে বলে **সুপারভাইজড লার্নিং** (supervised learning) — যেন একজন শিক্ষক পাশে বসে প্রতিবার বলে দিচ্ছেন উত্তরটা ঠিক হলো কি না।

যেহেতু উত্তরগুলো নির্দিষ্ট কয়েকটি ভাগে (৬টি) সীমাবদ্ধ — কোনো সংখ্যা নয় — একে বলে **ক্লাসিফিকেশন** (classification)। আরও নির্দিষ্টভাবে **multi-class, single-label**: ছয়টি শ্রেণি আছে, কিন্তু প্রতিটি বাক্য ঠিক একটিতেই পড়বে।

৩.৩ ডেটা তিন ভাগে ভাগ করা — কেন এটি অপরিহার্য

এটি পুরো প্রকল্পের সবচেয়ে গুরুত্বপূর্ণ ধারণাগুলোর একটি, এবং সুপারভাইজার প্রায় নিশ্চিতভাবে এটি জিজ্ঞেস করবেন।

ভাগ	পরিমাণ	কাজ
Train (প্রশিক্ষণ)	১৬,০০০	মডেল এখান থেকে শেখে — এর উপরেই ওজন সমন্বয় হয়
Validation (যাচাই)	২,০০০	ট্রেনিং চলাকালে কোন সংস্করণটি সেরা তা বাছাই করতে
Test (পরীক্ষা)	২,০০০	একদম শেষে, একবার — চূড়ান্ত ফলাফল ঘোষণার জন্য

পরীক্ষার হলের উপমা

- **Train** = পাঠ্যবই ও অনুশীলনী। শিক্ষার্থী বারবার পড়ে ও অনুশীলন করে।
- **Validation** = মডেল টেস্ট। কোন পড়ার কৌশলটি ভালো কাজ করছে তা বোঝার জন্য।
- **Test** = চূড়ান্ত পরীক্ষা। প্রশ্নপত্র **কখনোই** আগে দেখা হয়নি।

পরীক্ষার প্রশ্ন আগে দেখে ফেললে ১০০% পাওয়া সহজ — কিন্তু সেই নম্বর দিয়ে বোঝা যায় না শিক্ষার্থী আসলে কতটুকু শিখেছে। ঠিক এই কারণেই **টেস্ট সেট ট্রেনিং-এর সময় কখনো স্পর্শ করা হয় না**।

৩.৪ ওভারফিটিং — সবচেয়ে বড় বিপদ

ওভারফিটিং (overfitting) হলো মডেলের ট্রেনিং ডেটা **মুখস্থ** করে ফেলা, বোঝার বদলে। এমন মডেল ট্রেনিং সেটে ৯৯% পায়, কিন্তু নতুন বাক্যে ভয়াবহ ফল করে।

যেভাবে বুঝবেন: ট্রেনিং-এর স্কোর বাড়ছে কিন্তু ভ্যালিডেশনের স্কোর **কমছে** — এটিই ওভারফিটিং শুরুর লক্ষণ।

এই প্রকল্পে চারটি প্রতিরোধ ব্যবস্থা নেওয়া হয়েছে:

1. **প্রতি epoch-এ ভ্যালিডেশন যাচাই** — সমস্যা শুরু হলেই ধরা পড়ে।
2. **সেরা চেকপয়েন্ট নির্বাচন** (`load_best_model_at_end=True`) — শেষেরটি নয়, সবচেয়ে ভালোটি রাখা হয়।
3. **Weight decay = 0.01** — ওজনগুলোকে অতিরিক্ত বড় হতে দেয় না।
4. **অল্প epoch (৫)** — ফাইন-টিউনিং-এ বেশি epoch মানেই বেশি মুখস্থ।

৪. ডেটাসেট — dair-ai/emotion

৪.১ পরিচিতি

নাম	dair-ai/emotion (split কনফিগ)
উৎস গবেষণা	Saravia et al., "CARER: Contextualized Affect Representations for Emotion Recognition", EMNLP 2018
ধরন	ইংরেজি টুইট, প্রতিটিতে একটি করে আবেগের লেবেল
মোট আকার	২০,০০০ বাক্য (১৬,০০০ + ২,০০০ + ২,০০০)
শ্রেণি	০=sadness, ১=joy, ২=love, ৩=anger, ৪=fear, ৫=surprise
লাইসেন্স	গবেষণা ও শিক্ষামূলক ব্যবহারের জন্য

৪.২ শ্রেণি-বণ্টন — এবং এর গুরুতর প্রভাব

ট্রেনিং সেটের প্রকৃত হিসাব (আমাদের নিজের চালানো বিশ্লেষণ থেকে):

আবেগ	সংখ্যা	শতাংশ	দৃশ্যমান তুলনা
joy (আনন্দ)	৫,৩৬২	৩৩.৫১%	
sadness (দুঃখ)	৪,৬৬৬	২৯.১৬%	
anger (রাগ)	২,১৫৯	১৩.৪৯%	
fear (ভয়)	১,৯৩৭	১২.১১%	
love (ভালোবাসা)	১,৩০৪	৮.১৫%	
surprise (বিস্ময়)	৫৭২	৩.৫৮%	

শ্রেণি ভারসাম্যহীনতা (Class Imbalance) — ৯.৪ গুণ

সবচেয়ে বেশি (joy ৩৩.৫১%) আর সবচেয়ে কম (surprise ৩.৫৮%) শ্রেণির অনুপাত ৯.৪ গুণ। এর ভয়ানক পরিণতি একটি উদাহরণে বোঝা যায়:

ধরুন একটি "বোকা" মডেল বানালাম যেটি **সবকিছুকেই joy বলে** — কোনো শেখা ছাড়াই। সে ৩৩.৫% accuracy পাবে। আরও চতুর হয়ে যদি শুধু joy আর sadness-এর মধ্যে বাছাই করে, ৬২% পর্যন্ত যেতে পারে — **surprise শ্রেণিটি সম্পূর্ণ উপেক্ষা করেও**।

অর্থাৎ শুধু accuracy দেখে মডেলের মান বিচার করা বিপজ্জনক। এই কারণেই এই প্রকল্পে সেরা মডেল বাছাই করা হয় **macro-F1** দিয়ে, accuracy দিয়ে নয়। কেন — তা ১০ নং অধ্যায়ে ব্যাখ্যা করা আছে।

৪.৩ ডেটা পরিষ্কারকরণ — যা করা হয়নি এবং কেন

ডেটাসেটটি আগে থেকেই ছোট হাতের অক্ষরে (lowercase) রূপান্তরিত এবং যতিচিহ্নমুক্ত। তাই ভারী পরিষ্কারকরণের প্রয়োজন নেই — শুধু সামনে-পেছনের বাড়তি ফাঁকা জায়গা (whitespace) সরানো হয়েছে।

ইচ্ছাকৃতভাবে যা করা হয়নি: stopword মুছে ফেলা হয়নি। কারণ "i", "not", "am" — এই শব্দগুলোই আবেগের জন্য অত্যন্ত গুরুত্বপূর্ণ। "I am not happy" থেকে "not" সরালে অর্থ সম্পূর্ণ উল্টে যায়। ঐতিহ্যগত টেক্সট প্রসেসিং-এ stopword সরানো হয়, কিন্তু আবেগ শনাক্তকরণে সেটি ক্ষতিকর।

৫. টেক্সটকে সংখ্যায় রূপান্তর

নিউরাল নেটওয়ার্ক শুধু সংখ্যা নিয়ে কাজ করে। তাই "i feel happy" বাক্যটিকে সংখ্যায় রূপান্তর করতে হবে। এটি দুই ধাপে হয়।

৫.১ ধাপ ১ — টোকেনাইজেশন

টোকেনাইজেশন (tokenization) মানে বাক্যকে ছোট ছোট টুকরায় ভাগ করা এবং প্রতিটি টুকরাকে একটি সংখ্যা (ID) দেওয়া।

DistilBERT ব্যবহার করে **WordPiece** পদ্ধতি — এটি শব্দের চেয়েও ছোট অংশে ভাগতে পারে। এর সুবিধা হলো **অজানা শব্দের সমস্যা** (out-of-vocabulary) দূর হয়।

মূল বাক্য: "i feel unbelievably happy"

শব্দে ভাঙা: [i] [feel] [unbelievably] [happy]
↑ "unbelievably" অভিধানে না থাকলে সমস্যা!

WordPiece: [i] [feel] [un] [##bel] [##ie] [##va] [##bly] [happy]
↑ চেনা টুকরায় ভেঙে ফেলে – কোনো শব্দই অজানা থাকে না
(## চিহ্ন মানে এটি আগের টুকরার সাথে জোড়া)

ID-তে রূপান্তর:

[CLS]	i	feel	un	##bel	##ie	##va	##bly	happy	[SEP]
101	1045	2514	4895	8671	2666	3567	6321	3407	102

দুটি বিশেষ টোকেন যোগ হয়:

- **[CLS]** — শুরুতে বসে। ট্রেনিং শেষে এই টোকেনের ভেক্টরটি **পুরো বাক্যের সারসংক্ষেপ** ধারণ করে। শ্রেণিবিন্যাসের সিদ্ধান্ত এই একটি ভেক্টর থেকেই নেওয়া হয় — এটি খুব গুরুত্বপূর্ণ, মনে রাখুন।
- **[SEP]** — শেষে বসে, বাক্যের সমাপ্তি বোঝায়।

max_length = 96 কেন?

সব বাক্যের দৈর্ঘ্য সমান নয়, কিন্তু নিউরাল নেটওয়ার্কে একসাথে প্রক্রিয়াকরণের জন্য সমান দৈর্ঘ্য দরকার। আমরা সর্বোচ্চ ৯৬ টোকেন ধরেছি। আমাদের বিশ্লেষণে দেখা গেছে ট্রেনিং বাক্যের গড় দৈর্ঘ্য মাত্র ~১৯ শব্দ, এবং ৯৯% বাক্যই ৯৬ টোকেনের নিচে। অর্থাৎ প্রায় কোনো তথ্যই কাটা পড়ছে না, অথচ হিসাব অনেক দ্রুত হচ্ছে।

Dynamic Padding — একটি কার্যকর কৌশল

সব বাক্যকে ৯৬-এ টেনে লম্বা করলে বিপুল অপচয় হয় — ২০ টোকেনের বাক্যে ৭৬টি অর্থহীন শূন্য যোগ হয়। তাই ব্যবহার করা হয়েছে `DataCollatorWithPadding`, যা প্রতিটি ব্যাচে সেই ব্যাচের সবচেয়ে লম্বা বাক্য পর্যন্ত padding দেয়।

ফলাফল: ট্রেনিং প্রায় **দ্বিগুণ দ্রুত**, কোনো নির্ভুলতা না হারিয়ে। এটি সুপারভাইজারকে বলার মতো একটি ভালো অপটিমাইজেশন।

৫.২ ধাপ ২ — এমবেডিং

ID শুধু একটি নম্বর, এর কোনো অর্থ নেই — "happy" যদি ৩৪০৭ হয় আর "sad" যদি ৬২৭৪, তার মানে এই নয় যে sad, happy-এর চেয়ে বড়।

তাই প্রতিটি ID-কে একটি **ভেক্টর**-এ রূপান্তর করা হয় — DistilBERT-এ ৭৬৮টি সংখ্যার একটি তালিকা। একেই বলে **এমবেডিং** (embedding)।

এই ৭৬৮টি সংখ্যা শব্দের **অর্থ** ধারণ করে। জাদুটা হলো — **কাছাকাছি অর্থের শব্দ কাছাকাছি ভেক্টর পায়**। "happy" আর "joyful"-এর ভেক্টর প্রায় একই দিকে নির্দেশ করে, কিন্তু "angry"-র ভেক্টর অন্য দিকে। মডেল এই সম্পর্কগুলো ট্রেনিং থেকেই শিখেছে, কেউ হাতে লিখে দেয়নি।

৬. Transformer, BERT ও DistilBERT

৬.১ Attention — মূল ধারণা

২০১৭ সালের "Attention Is All You Need" গবেষণাপত্রে Transformer স্থাপত্য প্রস্তাব করা হয়। এর প্রাণভোমরা হলো **self-attention**।

মূল ধারণাটি সরল: **একটি শব্দের অর্থ নির্ভর করে বাক্যের অন্য কোন শব্দগুলোর উপর** — এবং মডেল নিজেই ঠিক করে নেয় কোন শব্দের দিকে কতটা "মনোযোগ" দেবে।

বাক্য: "I am not happy about this"

"happy" শব্দটি প্রক্রিয়াকরণের সময় মডেলের মনোযোগ বণ্টন:

I		৫%	
am		৫%	
not		৬০%	← সবচেয়ে বেশি মনোযোগ!
happy		২০%	
about		৫%	
this		৫%	

ফলে মডেল বোঝে: এখানে "happy" আসলে ইতিবাচক নয়, কারণ ঠিক আগেই "not" আছে।

পুরনো পদ্ধতি (যেমন TF-IDF) শুধু গোনে কোন শব্দ কতবার আছে — "not" আর "happy" যে পাশাপাশি, সেটি ধরতে পারে না। এখানেই Transformer-এর মূল শ্রেষ্ঠত্ব।

"Multi-head attention" মানে একসাথে অনেকগুলো (DistilBERT-এ ১২টি) ভিন্ন দৃষ্টিকোণ থেকে মনোযোগ দেওয়া — একটি head হয়তো ব্যাকরণ দেখে, আরেকটি আবেগ দেখে।

৬.২ BERT এবং প্রি-ট্রেনিং

BERT (Bidirectional Encoder Representations from Transformers) গুগলের তৈরি। এর বিশেষত্ব হলো এটি বাক্য **দুই দিক থেকেই** পড়ে — বাম থেকে ডান এবং ডান থেকে বাম একসাথে।

BERT-কে প্রথমে বিপুল পরিমাণ সাধারণ টেক্সটে (সম্পূর্ণ Wikipedia + হাজারো বই, প্রায় ৩৩০ কোটি শব্দ) ট্রেন করা হয়েছে একটি চতুর কাজ দিয়ে — **Masked Language Modeling**:

ইনপুট: "The cat sat on the [MASK]"
কাজ: [MASK] জায়গায় কোন শব্দ হবে অনুমান করো
উত্তর: "mat"

এভাবে কোটি কোটি বাক্যে অনুশীলন করতে করতে মডেল শিখে ফেলে:

- ব্যাকরণ ও বাক্যগঠন
- শব্দের অর্থ ও পারস্পরিক সম্পর্ক
- সাধারণ জ্ঞান ও প্রেক্ষাপট

এর বিশাল সুবিধা: এই ট্রেনিং-এ **কোনো মানুষের লেবেল লাগে না** — বাক্য থেকেই শব্দ লুকিয়ে দিলেই হলো। একে বলে **self-supervised learning**। এই পর্যায়টিকে বলে **প্রি-ট্রেনিং** (pre-training), যা গুগল করেছে, আমরা নয় — এতে লক্ষ লক্ষ টাকার GPU সময় লাগে।

৬.৩ DistilBERT — ছোট, দ্রুত, প্রায় সমান ভালো

BERT-base-এ ১১ কোটি প্যারামিটার — যা ফ্রি হোস্টিং-এ চালানো ব্যয়বহুল ও ধীর। **DistilBERT** হলো BERT-এর একটি সংকুচিত সংস্করণ, যা **Knowledge Distillation** পদ্ধতিতে তৈরি।

পদ্ধতিটি অনেকটা শিক্ষক-ছাত্রের মতো: বড় BERT ("শিক্ষক") যে উত্তর দেয়, ছোট মডেল ("ছাত্র") সেই উত্তর *এবং শিক্ষকের আত্মবিশ্বাসের ধরনটিও* নকল করতে শেখে। শুধু সঠিক উত্তর নয়, শিক্ষক কতটা দ্বিধায় ছিল সেটিও শেখে — এতে অনেক বেশি তথ্য পাওয়া যায়।

বৈশিষ্ট্য	BERT-base	DistilBERT
Transformer স্তর	১২	৬
প্যারামিটার	১১ কোটি	৬.৭ কোটি
আকার	~৪৪০ MB	~২৬৫ MB
গতি	১x	~১.৬x দ্রুত
কর্মদক্ষতা ধরে রাখে	১০০%	~৯৭%

কেন এটি বেছে নেওয়া হলো: ফ্রি হোস্টিং-এ RAM মাত্র ৫১২ MB এবং কোনো GPU নেই। BERT-base সেখানে চলবে না বা অসহনীয় ধীর হবে। DistilBERT ৩% নির্ভুলতা ছেড়ে দিয়ে ৪০% আকার ও ৬০% সময় বাঁচায় — এটি একটি **সচেতন প্রকৌশল সমঝোতা** (engineering trade-off), যা সুপারভাইজারকে ব্যাখ্যা করার মতো।

৭. বেসলাইন মডেল — TF-IDF + Logistic Regression

৭.১ বেসলাইন কেন দরকার?

সরাসরি DistilBERT ট্রেন করে ৯২% পেলে সেটি শুনতে দারুণ। কিন্তু একটি প্রশ্ন থেকে যায় — **একটি অতি সাধারণ পদ্ধতি কত পেত?** যদি সাধারণ পদ্ধতিও ৯১% পায়, তবে বিশাল ট্রান্সফরমারের পেছনে খরচ ও জটিলতা ন্যায্য নয়। **বেসলাইন** হলো সেই তুলনার মানদণ্ড। এটি গবেষণা-পদ্ধতির একটি মৌলিক অংশ, এবং **এটি আপনি নিজে ট্রেন করেছেন।**

৭.২ TF-IDF কী?

TF-IDF (Term Frequency – Inverse Document Frequency) প্রতিটি শব্দের গুরুত্ব মাপে দুটি বিষয় বিবেচনা করে:

- TF** — এই বাক্যে শব্দটি কতবার আছে? বেশি থাকলে বেশি গুরুত্বপূর্ণ।
- IDF** — শব্দটি সব বাক্যে কতটা সাধারণ? "the" সবখানে আছে, তাই এর গুরুত্ব কম। "furious" বিরল, তাই এটি পাওয়া গেলে সেটি অনেক তথ্যবহুল।

আমরা ব্যবহার করেছি `ngram_range=(1,2)` — অর্থাৎ একক শব্দ এবং পাশাপাশি দুই শব্দের জোড়াও গোনা হয়। এতে "not happy" জোড়াটি একটি আলাদা বৈশিষ্ট্য হিসেবে ধরা পড়ে, যা কিছুটা প্রেক্ষাপট রক্ষা করে। সর্বোচ্চ ২০,০০০টি বৈশিষ্ট্য রাখা হয়েছে।

৭.৩ ফলাফল (নিজে চালানো)

ডেটা ভাগ	Accuracy	Macro-F1
Validation	৮৭.৯০%	০.৮৫১০
Test	৮৭.১৫%	০.৮৩৪৪

ট্রেনিং সময় মাত্র **১৫.৩ সেকেন্ড** — সাধারণ ল্যাপটপের CPU-তে। তুলনায় DistilBERT ফাইন-টিউনিং-এ GPU-তে ~৬ মিনিট, CPU-তে ~৪৫ মিনিট লাগে।

বেসলাইনের দ্বিতীয় ব্যবহার — একটি স্মার্ট ডিজাইন

বেসলাইন মডেলটি ফেলে দেওয়া হয়নি। এটি ব্যাকএন্ডে **বিকল্প ব্যবস্থা** (fallback) হিসেবে রাখা আছে। যদি কোনো কারণে বড় ট্রান্সফরমার মডেল লোড না হয় (ইন্টারনেট সমস্যা, মেমরি সংকট, ভুল ঠিকানা), API সম্পূর্ণ বন্ধ না হয়ে বেসলাইন দিয়ে উত্তর দেওয়া চালিয়ে যায় — এবং প্রতিটি উত্তরে `model_source: "baseline"` লিখে **সংভাবে জানিয়ে দেয়** যে এটি দুর্বল মডেলের ফলাফল।

একে বলে **graceful degradation** — সম্পূর্ণ বিকল হওয়ার চেয়ে সীমিত সেবা ভালো।

৮. মূল ট্রেনিং — Fine-tuning (সবচেয়ে গুরুত্বপূর্ণ অধ্যায়)

৮.১ Transfer Learning ও Fine-tuning কী?

আমরা DistilBERT-কে **শূন্য থেকে** ট্রেন করি না। সেটি করতে কোটি টাকার হার্ডওয়্যার আর সপ্তাহের পর সপ্তাহ লাগত।

বরং আমরা গুগলের **আগে থেকে ট্রেন করা** মডেল নিই — যেটি ইতিমধ্যে ইংরেজি ভাষা জানে — এবং কেবল আমাদের নির্দিষ্ট কাজের জন্য সামান্য সমন্বয় করি। একেই বলে **Transfer Learning**, আর সমন্বয়ের প্রক্রিয়াটিকে বলে **Fine-tuning**।

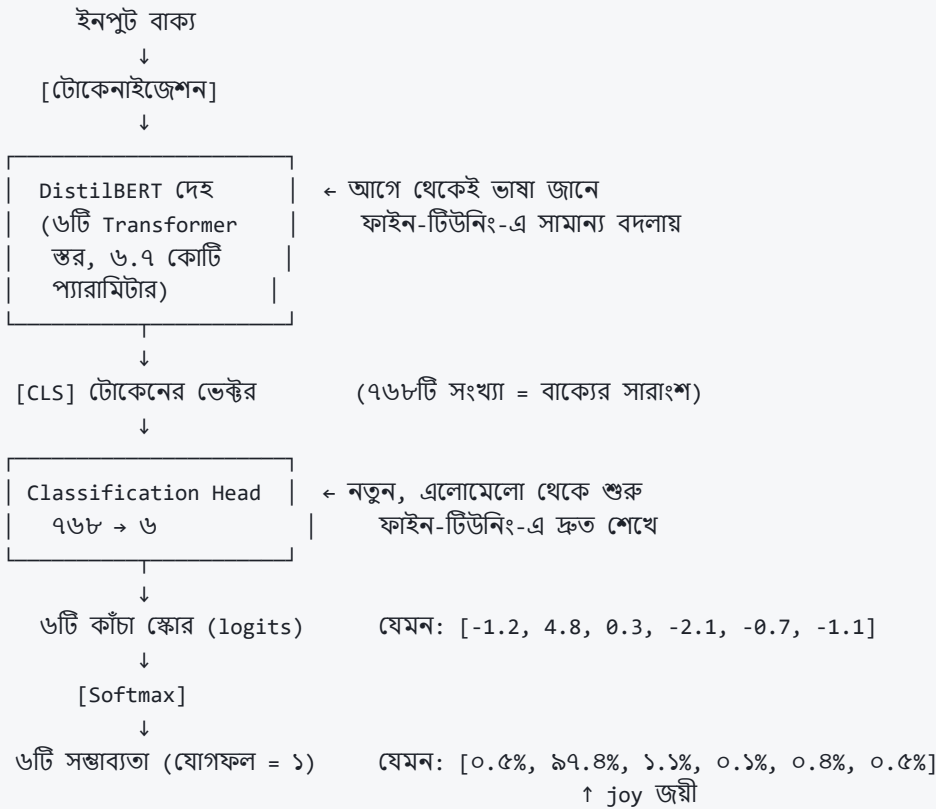
উপমা

একজন মানুষ ইতিমধ্যে বাংলা ভাষা জানেন (= প্রি-ট্রেনিং, বহু বছরের)। এখন তাঁকে "চিকিৎসা রিপোর্ট পড়া" শেখাতে হলে আবার প্রথম থেকে ভাষা শেখাতে হবে না — কয়েক সপ্তাহের বিশেষ প্রশিক্ষণই যথেষ্ট (= ফাইন-টিউনিং)।

৮.২ স্থাপত্য পরিবর্তন — Classification Head যোগ করা

প্রি-ট্রেন করা DistilBERT-এর কাজ ছিল "লুকানো শব্দ অনুমান করা" — এর আউটপুট স্তরে ৩০,৫২২টি সম্ভাব্য শব্দ। আমাদের দরকার মাত্র ৬টি আবেগ। তাই:

১. পুরনো আউটপুট স্তরটি **ফেলে দেওয়া হয়**।
২. জায়গায় একটি নতুন ছোট স্তর (**classification head**) বসানো হয়, যার আউটপুট ৬টি।
৩. এই নতুন স্তরের ওজন **এলোমেলোভাবে** শুরু হয় — সে কিছুই জানে না।



৮.৩ Softmax — কাঁচা স্কোরকে সম্ভাব্যতায় রূপান্তর

মডেলের সরাসরি আউটপুট (**logits**) যেকোনো সংখ্যা হতে পারে — ঋণাত্মকও। এগুলোকে সম্ভাব্যতায় (০ থেকে ১, যোগফল ১) বদলাতে **Softmax** ব্যবহার হয়:

$$P(\text{শ্রেণি } i) = e^{(z_i)} / \sum e^{(z_j)}$$

প্রতিটি স্কোরকে e -এর ঘাতে বসানো হয় (সব ধনাত্মক হয়ে যায়),
তারপর মোট যোগফল দিয়ে ভাগ (যোগফল ১ হয়ে যায়)।
বড় স্কোর অসামঞ্জস্যপূর্ণভাবে বেশি সুবিধা পায়।

এই সম্ভাব্যতাগুলোই ফ্রন্টএন্ডের বার-চার্ট ও রাডার চার্টে দেখানো হয়।

৮.৪ Loss Function — ভুল মাপার নিয়ম

শেখার জন্য মডেলকে জানতে হবে সে **কতটা ভুল** করেছে। এই পরিমাপকে বলে **loss**। আমরা ব্যবহার করি **Cross-Entropy Loss**:

$$\text{Loss} = -\log(P(\text{সঠিক শ্রেণি}))$$

সঠিক উত্তরে মডেলের আস্থা	Loss	ব্যাখ্যা
৯৯% (প্রায় নিশ্চিত ও সঠিক)	০.০১	প্রায় শাস্তি নেই
৫০% (দ্বিধাগ্রস্ত)	০.৬৯	মাঝারি শাস্তি
১০% (ভুল করেছে)	২.৩০	ভারী শাস্তি
১% (আত্মবিশ্বাসের সাথে ভুল)	৪.৬১	অত্যন্ত ভারী শাস্তি

লক্ষ করুন — আত্মবিশ্বাসের সাথে ভুল করাকে সবচেয়ে কঠোরভাবে শাস্তি দেওয়া হয়। এটি ইচ্ছাকৃত: আমরা চাই মডেল নিশ্চিত না হলে দ্বিধা প্রকাশ করুক।

৮.৫ ট্রেনিং লুপ — একটি ধাপে কী ঘটে

এটি ট্রেনিং-এর হৃদপিণ্ড। প্রতিটি ধাপে পাঁচটি কাজ হয়:

একটি ট্রেনিং ধাপ (one step) – ৩২টি বাক্যের একটি ব্যাচ

- FORWARD PASS** (সামনের দিকে হিসাব)
৩২টি বাক্য মডেলে ঢেকানো হয়
→ প্রতিটির জন্য ৬টি করে সম্ভাব্যতা বেরিয়ে আসে
 - LOSS গণনা**
ভবিষ্যদ্বাণী বনাম আসল উত্তর তুলনা
→ একটি সংখ্যা: "মডেল এই ব্যাচে কতটা ভুল করেছে"
 - BACKWARD PASS** (Backpropagation)
ক্যালকুলাসের চেইন রুল ব্যবহার করে হিসাব করা হয়:
"৬.৭ কোটি প্যারামিটারের প্রত্যেকটি এই ভুলে কতটা দায়ী?"
→ প্রতিটির জন্য একটি gradient (ঢাল) পাওয়া যায়
 - OPTIMIZER আপডেট** (AdamW)
প্রতিটি প্যারামিটারকে ভুল কমানোর দিকে সামান্য সরানো হয়
নতুন ওজন = পুরনো ওজন - (learning_rate × gradient)
 - GRADIENT রিসেট**
পরের ব্যাচের জন্য ঢালগুলো শূন্য করা হয়
- পরের ৩২টি বাক্য নিয়ে আবার ১ থেকে শুরু

৮.৬ Epoch, Batch, Step — হিসাবটি বুঝে নিন

পরিভাষা	অর্থ	আমাদের হিসাব
Batch	একসাথে প্রক্রিয়াকৃত বাক্যের সংখ্যা	৩২টি বাক্য
Step	একটি ব্যাচ নিয়ে একবার ওজন আপডেট	$১৬০০০ \div ৩২ = ৫০০ \text{ step/epoch}$
Epoch	পুরো ট্রেনিং ডেটা একবার সম্পূর্ণ দেখা	৫টি epoch
মোট	সম্পূর্ণ ট্রেনিং-এ মোট আপডেট	$৫০০ \times ৫ = ২,৫০০ \text{ step}$

অর্থাৎ ওজনগুলো ২,৫০০ বার সংশোধিত হয়, এবং মডেল প্রতিটি বাক্য মোট ৫ বার দেখে।

৮.৭ প্রতিটি Epoch শেষে যা ঘটে

1. ট্রেনিং সাময়িক থামে; মডেলকে **evaluation mode**-এ নেওয়া হয়।
2. ২,০০০টি **ভ্যালিডেশন** বাক্যে ভবিষ্যদ্বাণী করা হয় (এগুলো থেকে কোনো শেখা হয় না)।
3. Accuracy ও macro-F1 হিসাব করা হয়।
4. মডেলের বর্তমান অবস্থা (**checkpoint**) ডিস্কে সংরক্ষিত হয়।
5. ট্রেনিং আবার শুরু হয়।

সব epoch শেষে `load_best_model_at_end=True` সেটিংটির কারণে **সর্বোচ্চ ভ্যালিডেশন macro-F1 পাওয়া চেকপয়েন্টটি** ফিরিয়ে আনা হয় — শেষেরটি নয়। কারণ শেষ epoch-এ ওভারফিটিং শুরু হয়ে থাকতে পারে।

৯. হাইপারপ্যারামিটার — প্রতিটির অর্থ ও যুক্তি

হাইপারপ্যারামিটার হলো সেই সেটিংস যা আমরা ঠিক করে দিই ট্রেনিং শুরুর আগে। (এর বিপরীতে প্যারামিটার বা ওজন — যা মডেল নিজে শেখে।) সুপারভাইজার প্রায় নিশ্চিতভাবে এগুলোর ব্যাখ্যা চাইবেন।

সেটিং	মান	কেন এই মান
learning_rate	2e-5	প্রতি ধাপে ওজন কতটা বদলাবে। খুব বড় হলে সেরা সমাধান পেরিয়ে যায় এবং প্রি-ট্রেনিং-এর জ্ঞান নষ্ট হয় (catastrophic forgetting)। খুব ছোট হলে শেখা অসম্ভব ধীর। ফাইন-টিউনিং-এ 2e-5 থেকে 5e-5 প্রমাণিত পরিসর।
batch_size	32	একসাথে কতগুলো বাক্য। বড় ব্যাচে ঢালের হিসাব স্থিতিশীল ও দ্রুত হয়, কিন্তু বেশি GPU মেমরি লাগে। ৩২ হলো Kaggle-এর T4 GPU-র (১৬GB) জন্য নিরাপদ ও কার্যকর মান।
num_epochs	5	পুরো ডেটা কতবার দেখা হবে। ফাইন-টিউনিং-এ ৩-৫ যথেষ্ট; এর বেশি হলে ওভারফিটিং শুরু হয়। সেরা চেকপয়েন্ট নির্বাচন থাকায় বাড়তি epoch ক্ষতি করে না।
weight_decay	0.01	একটি regularization কৌশল — ওজনগুলোকে অতিরিক্ত বড় হতে বাধা দেয়। বড় ওজন মানে মডেল কয়েকটি নির্দিষ্ট বৈশিষ্ট্যের উপর অতি-নির্ভরশীল, যা ওভারফিটিং-এর লক্ষণ।
warmup_ratio	0.06	প্রথম ৬% ধাপে learning rate ০ থেকে ধীরে ধীরে বাড়ানো হয়। শুরুতে classification head এলোমেলো, তাই তখনকার ঢালগুলো বিশৃঙ্খল — সেই সময় বড় পদক্ষেপ নিলে প্রি-ট্রেন করা ভালো ওজনগুলো নষ্ট হয়ে যেতে পারে।
max_length	96	সর্বোচ্চ টোকেন সংখ্যা। ডেটার ৯৯%-এর বেশি এর নিচে, তাই তথ্য হারানো নগণ্য, কিন্তু হিসাব অনেক হালকা।
optimizer	AdamW	Adam-এর উন্নত সংস্করণ, যা weight decay সঠিকভাবে প্রয়োগ করে। Transformer ট্রেনিং-এ কার্যত মানদণ্ড।
fp16	GPU-তে চালু	৩২-বিটের বদলে ১৬-বিট দশমিক সংখ্যা ব্যবহার। প্রায় দ্বিগুণ দ্রুত ও অর্ধেক মেমরি, নির্ভুলতায় প্রভাব নগণ্য। CPU-তে সুবিধা নেই তাই স্বয়ংক্রিয়ভাবে বন্ধ থাকে।
seed	42	এলোমেলোতা নিয়ন্ত্রণ করে, যাতে একই কোড আবার চালালে একই ফলাফল পাওয়া যায় — বৈজ্ঞানিক পুনরুৎপাদনযোগ্যতার জন্য অপরিহার্য।
metric_for_best_model	f1_macro	সবচেয়ে গুরুত্বপূর্ণ সিদ্ধান্ত। accuracy দিয়ে বাছাই করলে যে মডেল বিরল surprise শ্রেণিটি প্রায় উপেক্ষা করে সেও জিতে যেত। macro-F1 সব শ্রেণিকে সমান গুরুত্ব দেয়, তাই বিরল শ্রেণিকে সুরক্ষা দেয়।

৯.১ শ্রেণি ভারসাম্যহীনতার বিকল্প সমাধান

কোডে `--class-weights` নামে একটি সুইচ রাখা আছে। এটি চালু করলে **বিরল শ্রেণির ভুলকে বেশি শাস্তি** দেওয়া হয় — surprise ভুল করলে joy ভুল করার চেয়ে অনেক বেশি loss যোগ হয়।

কিন্তু এটি ডিফল্টভাবে বন্ধ রাখা হয়েছে, কারণ সাধারণত ফাইন-টিউনিং একাই ভারসাম্যহীনতা সামলে নেয়, এবং জোর করে ওজন দিলে সামগ্রিক নির্ভুলতা কমে যেতে পারে। প্রথমে ফলাফল দেখা, তারপর প্রয়োজন হলে ব্যবহার করা — এটিই সঠিক পদ্ধতি। এটিও সুপারভাইজারকে বলার মতো একটি বিবেচিত সিদ্ধান্ত।

১০. মূল্যায়ন — মেট্রিক বোঝা

১০.১ চারটি সম্ভাব্য ফলাফল

একটি নির্দিষ্ট শ্রেণির (ধরুন joy) দৃষ্টিকোণ থেকে প্রতিটি ভবিষ্যদ্বাণী চার ভাগের একটিতে পড়ে:

	মডেল বলেছে joy	মডেল বলেছে joy নয়
আসলে joy	TP — সত্য ইতিবাচক	FN — মিথ্যা নেতিবাচক (মিস করেছে)
আসলে joy নয়	FP — মিথ্যা ইতিবাচক (ভুল অ্যালার্ম)	TN — সত্য নেতিবাচক

১০.২ চারটি মেট্রিক

Accuracy (নির্ভুলতা)

Accuracy = (মোট সঠিক ভবিষ্যদ্বাণী) / (মোট ভবিষ্যদ্বাণী)

সহজবোধ্য, কিন্তু ভারসাম্যহীন ডেটায় **বিদ্রাস্তিকর** — কারণ এটি বড় শ্রেণিগুলোর দ্বারা প্রভাবিত হয়। ৪.২ অনুচ্ছেদের "বোকা মডেল" উদাহরণটি মনে করুন।

Precision (নির্ভুলতা/শুদ্ধতা)

Precision = TP / (TP + FP)

"মডেল যতবার 'joy' বলেছে, তার কত ভাগ আসলেই joy ছিল?"

কম precision = মডেল অতিরিক্ত উৎসাহী, প্রায়ই ভুল অ্যালার্ম দেয়।

Recall (পুনরুদ্ধার/ধরার ক্ষমতা)

Recall = TP / (TP + FN)

"ডেটায় যতগুলো আসল joy ছিল, তার কত ভাগ মডেল খুঁজে পেয়েছে?"

কম recall = মডেল অনেকগুলো মিস করেছে।

Precision বনাম Recall — একটি টানাপোড়েন

একটি মাছ ধরার জালের কথা ভাবুন। **ছোট ফাঁকের জাল** সব মাছ ধরবে (উচ্চ recall) কিন্তু আবর্জনাও উঠবে (নিম্ন precision)। **বড় ফাঁকের জাল** শুধু বড় মাছ ধরবে (উচ্চ precision) কিন্তু অনেক মাছ পালাবে (নিম্ন recall)।

দুটো একসাথে বাড়ানো সাধারণত সম্ভব নয় — একটি বাড়লে অন্যটি কমে। তাই দুটোকে একত্রে মাপার একটি সংখ্যা দরকার।

F1-Score

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

এটি Precision ও Recall-এর **হারমোনিক গড়** — সাধারণ গড় নয়। পার্থক্যটি গুরুত্বপূর্ণ: হারমোনিক গড়ে **যেকোনো একটি খুব কম হলেই F1 কমে যায়**।

Precision	Recall	সাধারণ গড়	F1 (হারমোনিক)
১.০০	০.০২	০.৫১ 😊	০.০৪ 😞
০.৫০	০.৫০	০.৫০	০.৫০

প্রথম সারিতে মডেলটি কার্যত অকেজো — সাধারণ গড় সেটি লুকিয়ে ফেলে, কিন্তু F1 ধরে ফেলে।

১০.৩ Macro-F1 বনাম Weighted-F1 — প্রকল্পের কেন্দ্রীয় সিদ্ধান্ত

ছয়টি শ্রেণির আলাদা F1 আছে। এদের একত্রিত করার দুটি উপায়:

পদ্ধতি	হিসাব	প্রভাব
Macro-F1	৬টি F1-এর সরল গড়	প্রতিটি শ্রেণি সমান ওজন পায় — ৬৬টি নমুনার surprise আর ৬৯৫টি নমুনার joy সমান গুরুত্বপূর্ণ
Weighted-F1	নমুনা-সংখ্যা অনুযায়ী ভারিত গড়	বড় শ্রেণিগুলো প্রাধান্য পায় — বিরল শ্রেণির দুর্বলতা ঢাকা পড়ে

আমাদের ফলাফলে এই পার্থক্য স্পষ্ট দেখা যায়

$$\text{Macro-F1} = ০.৮৮২৫ \cdot \text{Weighted-F1} = ০.৯২৬৯$$

Weighted-F1 প্রায় ৪.৪ পয়েন্ট বেশি! কারণ এটি বড় শ্রেণিগুলোর ভালো ফলাফল দিয়ে surprise -এর দুর্বল ফলাফল (F1 মাত্র ০.৭৩৭৭) ঢেকে দিচ্ছে। **এই কারণেই আমরা macro-F1 রিপোর্ট করি এবং সেটি দিয়েই সেরা মডেল বাছাই করি** — এটি বেশি সৎ ও কঠোর মানদণ্ড।

১১. ফলাফল বিশ্লেষণ

১১.১ সামগ্রিক ফলাফল (টেস্ট সেট, ২,০০০ বাক্য)

মডেল	Accuracy	Macro-F1	Weighted-F1
TF-IDF + Logistic Regression (বেসলাইন)	৮৭.১৫%	০.৮৩৪৪	০.৮৭৪১
DistilBERT (ডিপ্লয় করা)	৯২.৭০%	০.৮৮২৫	০.৯২৬৯
পার্থক্য	+৫.৫৫ পয়েন্ট	+০.০৪৮১	+০.০৫২৮

উপসংহার: ট্রান্সফরমার বেসলাইনকে স্পষ্টভাবে হারিয়েছে — accuracy-তে ৫.৫ পয়েন্ট এবং macro-F1-এ ০.০৪৮। অর্থাৎ ট্রান্সফরমার ব্যবহারের জটিলতা **ন্যায্য**। বেসলাইন না থাকলে এই দাবিটি করা যেত না।

১১.২ শ্রেণিভিত্তিক ফলাফল

আবেগ	Precision	Recall	F1	নমুনা
sadness	০.৯৬৮৯	০.৯৬৫৬	০.৯৬৭২	৫৮১
joy	০.৯৪৯৩	০.৯৪২৪	০.৯৪৫৮	৬৯৫
anger	০.৯২৬৭	০.৯২০০	০.৯২৩৪	২৭৫
fear	০.৮৭৬১	০.৯১৫২	০.৮৯৫২	২২৪
love	০.৮০৩৬	০.৮৪৯১	০.৮২৫৭	১৫৯
surprise	০.৮০৩৬	০.৬৮১৮	০.৭৩৭৭	৬৬

সবচেয়ে গুরুত্বপূর্ণ পর্যবেক্ষণ

লক্ষ করুন — **নমুনা সংখ্যা যত কম, ফলাফল তত খারাপ**। এটি কাকতালীয় নয়, এটি সরাসরি শ্রেণি ভারসাম্যহীনতার পরিণতি:

- sadness — ৫৮১টি নমুনা → F1 ০.৯৬৭
- surprise — ৬৬টি নমুনা → F1 ০.৭৩৮

surprise -এর recall মাত্র **০.৬৮১৮** — অর্থাৎ ৬৬টি আসল বিষয়সূচক বাক্যের মধ্যে মডেল মাত্র ৪৫টি ধরতে পেরেছে, **২১টি মিস করেছে**। এটি প্রকল্পের সবচেয়ে বড় দুর্বলতা এবং এটি লুকানো হয়নি — অ্যাপের About পাতায় স্পষ্টভাবে দেখানো আছে।

১১.৩ Confusion Matrix — ভুলগুলো কোথায়?

Confusion Matrix দেখায় মডেল কোন শ্রেণিকে কোন শ্রেণির সাথে গুলিয়ে ফেলছে। সারি = আসল উত্তর, কলাম = মডেলের ভবিষ্যদ্বাণী। কর্ণ (তির্থক) বরাবরের সংখ্যাগুলো সঠিক।

আসল ↓ / বলেছে →	sadness	joy	love	anger	fear	surprise
sadness	৫৬১	৪	১	১০	৫	০
joy	১	৬৫৫	৩২	৩	০	৪
love	০	২৩	১৩৫	১	০	০
anger	১০	৩	০	২৫৩	৯	০
fear	৬	০	০	৬	২০৫	৭
surprise	১	৫	০	০	১৫	৪৫

তিনটি প্রধান ভুলের ধরন — এবং এগুলো যুক্তিসঙ্গত

- joy ↔ love গুলিয়ে ফেলা (৩২ + ২৩ = ৫৫টি ভুল)** — সবচেয়ে বড় সমস্যা। এটি বোধগম্য: "i feel so wonderful with him" — এটি আনন্দ না ভালোবাসা? **মানুষও একমত হবে না।** দুটি আবেগ প্রকৃতিগতভাবেই ঘনিষ্ঠ, এবং ভালোবাসাকে আনন্দের একটি উপ-শ্রেণি হিসেবেও দেখা যায়।
- surprise → fear (১৫টি ভুল)** — surprise-এর সবচেয়ে বড় ক্ষতির উৎস। বিশ্বয় প্রায়ই ভয়ের ভাষা ব্যবহার করে: "i cant believe this is happening" বাক্যটি আনন্দের বিশ্বয়ও হতে পারে, আতঙ্কও হতে পারে। শুধু শব্দ দেখে পার্থক্য করা কঠিন।
- anger ↔ sadness (১০ + ১০ = ২০টি ভুল)** — দুটোই নেতিবাচক আবেগ এবং প্রায়ই একই শব্দভাণ্ডার ব্যবহার করে।

এই বিশ্লেষণটি গুরুত্বপূর্ণ, কারণ এটি দেখায় মডেলের ভুলগুলো এলোমেলো নয় — **ভাষাগতভাবে যুক্তিসঙ্গত**। সুপারভাইজারকে এটি বলতে পারলে বোঝা যাবে আপনি শুধু সংখ্যা দেখেননি, ভুলের কারণও বুঝেছেন।

১১.৪ গতি

CPU-তে ২,০০০টি বাক্য মূল্যায়নে সময় লেগেছে **১০৬.১৪ সেকেন্ড**, অর্থাৎ প্রতি বাক্যে **৫৩ মিলিসেকেন্ড**। লাইভ সার্ভারে গরম অবস্থায় একটি অনুরোধে ~৭০-১৫০ মিলিসেকেন্ড লাগে, যা রিয়েল-টাইম অভিজ্ঞতার জন্য যথেষ্ট দ্রুত।

১২. ব্যাকএন্ড — FastAPI ইনফারেন্স সার্ভিস

১২.১ কাজ

ব্যাকএন্ডের দায়িত্ব: HTTP অনুরোধ গ্রহণ → মডেল চালানো → JSON উত্তর ফেরত পাঠানো।

মেথড	পথ	কাজ
POST	/api/predict	একটি বাক্য বিশ্লেষণ
POST	/api/predict/batch	সর্বোচ্চ ২০০টি বাক্য একসাথে + সমষ্টিগত বণ্টন
GET	/api/health	সার্ভার ও মডেলের অবস্থা
GET	/api/model	প্রকৃত মূল্যায়ন সংখ্যা
GET	/docs	স্বয়ংক্রিয় ইন্টারঅ্যাক্টিভ API ডকুমেন্টেশন

১২.২ চারটি গুরুত্বপূর্ণ ডিজাইন সিদ্ধান্ত

১. মডেল একবারই লোড হয়

মডেল লোড হতে ২০-৬০ সেকেন্ড লাগে। প্রতিটি অনুরোধে লোড করলে সিস্টেম অচল হয়ে যেত। তাই সার্ভার চালু হওয়ার সময় **একবার** লোড হয় এবং মেমরিতে থেকে যায়।

২. ব্যাকগ্রাউন্ড থ্রেডে লোডিং

মডেল লোডিং মূল প্রোগ্রামকে আটকে রাখে না — এটি একটি আলাদা থ্রেডে চলে। ফলে `/api/health` সাথে সাথেই উত্তর দিতে পারে `status: "loading"` বলে। এই সংকেতটি দেখেই ফ্রন্টএন্ড "মডেল জেগে উঠছে..." বার্তা দেখায় সাদা পর্দার বদলে।

৩. Graceful Degradation

ট্রান্সফরমার লোড না হলে সিস্টেম বন্ধ হয় না — বেসলাইন মডেলে নেমে আসে এবং প্রতিটি উত্তরে `model_source` ক্ষেত্রে সত্যটি লিখে দেয়।

৪. থ্রেডপুলে ইনফারেন্স

PyTorch-এর হিসাব **blocking** — এটি সরাসরি চালালে সার্ভারের event loop আটকে যেত এবং অন্য কোনো ব্যবহারকারী উত্তর পেতেন না। তাই `run_in_threadpool` দিয়ে আলাদা থ্রেডে চালানো হয়।

১২.৩ নিরাপত্তা ও সুরক্ষা

- **ইনপুট যাচাই** — খালি টেক্সট বা ১০০০ অক্ষরের বেশি হলে ৪২২ ত্রুটি।
- **CORS** — শুধুমাত্র নির্দিষ্ট ডোমেইন থেকে অনুরোধ গ্রহণ।
- **Rate limiting** — প্রতি IP থেকে মিনিটে ৬০টি অনুরোধ। তবে `/api/health` সীমার বাইরে, কারণ ফ্রন্টএন্ড নিয়মিত সেটি পরীক্ষা করে।
- **গোপনীয়তা** — ব্যবহারকারীর লেখা **কোথাও সংরক্ষণ করা হয় না**। লগে শুধু দৈর্ঘ্য, ফলাফল ও সময় রাখা হয়।

১৩. ফ্রন্টএন্ড — ব্যবহারকারী ইন্টারফেস

১৩.১ চারটি পাতা

পাতা	বিবরণ
/ — হোম	স্বয়ংক্রিয়ভাবে টাইপ হওয়া লাইভ ডেমো, যা আসল API-তে অনুরোধ পাঠায়
/analyze	মূল কর্মক্ষেত্র — টাইপ থামালেই ৪০০ms পরে বিশ্লেষণ, রাডার চার্ট, বার চার্ট, সেশন ইতিহাস
/batch	অনেক লাইন বা CSV ফাইল একসাথে — সারণি, সমষ্টিগত চার্ট, CSV রপ্তানি
/about	ডেটাসেট, প্রকৃত মেট্রিক, সীমাবদ্ধতা

১৩.২ Debouncing — একটি গুরুত্বপূর্ণ কৌশল

ব্যবহারকারী টাইপ করার সময় প্রতিটি অক্ষরে API ডাকলে ২০ অক্ষরের বাক্যে ২০টি অনুরোধ যেত — সার্ভারের উপর অযথা চাপ।

Debouncing মানে: টাইপ থেমে যাওয়ার ৪০০ মিলিসেকেন্ড পরে একবারই অনুরোধ পাঠানো। এছাড়া নতুন অনুরোধ শুরু হলে আগেরটি **বাতিল** করা হয়, যাতে পুরনো ধীরগতির উত্তর এসে নতুন সঠিক উত্তরকে চাপা না দেয়।

১৩.৩ প্রতিটি অবস্থার নকশা

একটি পেশাদার অ্যাপ শুধু সফল অবস্থার কথা ভাবে না। এখানে প্রতিটি অবস্থা আলাদাভাবে ডিজাইন করা:

- **খালি** — উদাহরণ বাক্যের বোতাম দেওয়া থাকে
- **লোডিং** — ঘুরন্ত চাকা নয়, **skeleton** (কাঠামোর ছায়া), যাতে ফল এলে পাতা লাফিয়ে না ওঠে
- **জেগে ওঠা** — ফ্রি সার্ভারের ঠান্ডা শুরুর জন্য আলাদা বার্তা
- **ত্রুটি** — মানুষের ভাষায় ব্যাখ্যা + "আবার চেষ্টা করুন" বোতাম
- **সফল** — অ্যানিমেশনসহ চার্ট

১৩.৪ প্রবেশগম্যতা (Accessibility)

- রাডার চার্টের একটি **লিখিত বিকল্প** আছে স্ক্রিন রিডারের জন্য (চার্ট অঙ্ক ব্যবহারকারীর কাছে অর্থহীন)
- সব ইনপুটে যথাযথ **label**
- কীবোর্ড দিয়ে সম্পূর্ণ ব্যবহারযোগ্য
- **aria-live** দিয়ে অবস্থার পরিবর্তন ঘোষণা
- ব্যবহারকারী "কম অ্যানিমেশন" পছন্দ করলে সব চলাচল বন্ধ

১৪. ডিপ্লয়মেন্ট

অংশ	প্ল্যাটফর্ম	ঠিকানা
ফ্রন্টএন্ড	Vercel	emotionsense-ochre.vercel.app
ব্যাকএন্ড	Render (Docker)	emotionsense-api-452m.onrender.com
সোর্স কোড	GitHub	github.com/claudeproject751-dot/tricore

১৪.১ Docker কেন?

Docker পুরো পরিবেশটিকে (Python সংস্করণ, সব লাইব্রেরি, মডেল ফাইল) একটি **পাত্র** বেঁধে ফেলে। ফলে "আমার কম্পিউটারে তো চলছিল" সমস্যাটি দূর হয় — যেখানেই চালান, একই ফল।

দুটি অপ্টিমাইজেশন করা হয়েছে: (১) CPU-only PyTorch ব্যবহার — ২.৫ GB-র বদলে ২০০ MB; (২) মডেলের ওজন **ইমেজের ভেতরেই** রাখা, যাতে সার্ভার চালু হওয়ার সময় ২৫০ MB ডাউনলোড করতে না হয়।

১৪.২ Cold Start — ফ্রি হোস্টিং-এর বাস্তবতা

Render-এর ফ্রি সেবায় ১৫ মিনিট নিষ্ক্রিয় থাকলে সার্ভার ঘুমিয়ে পড়ে। পরের অনুরোধে জাগতে ৩০–৬০ সেকেন্ড লাগে।

এটি লুকানো হয়নি — বরং একটি **ডিজাইন করা অবস্থা** হিসেবে সামলানো হয়েছে: API সাথে সাথে ৫০৩ কোড ও **Retry-After** হেডার পাঠায়, আর ফ্রন্টএন্ড "মডেল জেগে উঠছে, ফ্রি সার্ভারে এক মিনিট পর্যন্ত লাগতে পারে" বার্তা দেখায়। **সমস্যাকে সংভাবে ব্যাখ্যা করাও একটি প্রকৌশল সমাধান।**

১৫. টেস্টিং ও CI

অংশ	সংখ্যা	কী পরীক্ষা করে
ব্যাকএন্ড (pytest)	৪০	সঠিক উত্তর, খালি ইনপুট, অতিরিক্ত বড় ইনপুট, ব্যাচ, cold start, CORS, fallback
ফ্রন্টএন্ড (Vitest)	১৮	API ত্রুটি ব্যবস্থাপনা, debounce, ফর্ম আচরণ, ইতিহাস সংরক্ষণ
মোট	৫৮	সবগুলো পাস

টেস্টে আসল মডেল ব্যবহার করা হয় না — একটি **নকল (stub)** মডেল ব্যবহার হয়। ফলে টেস্ট কয়েক সেকেন্ডে শেষ হয় এবং ২৫০ MB ডাউনলোড লাগে না।

CI (Continuous Integration): GitHub-এ কোড পাঠানো মাত্র স্বয়ংক্রিয়ভাবে সব টেস্ট, lint, টাইপ-চেক ও Docker বিল্ড চলে। কিছু ভাঙলে সাথে সাথে জানা যায়।

১৬. সীমাবদ্ধতা ও নৈতিক দিক

⚠ এটি কোনো চিকিৎসা বা রোগনির্ণয়ের যন্ত্র নয়

এই সিস্টেম শুধু **শব্দ দেখে** আবেগ অনুমান করে। এটি কখনোই মানসিক স্বাস্থ্য পরীক্ষা, চাকরির নিয়োগ, বা কোনো ব্যক্তির অধিকার ও কল্যাণ সংক্রান্ত সিদ্ধান্তে ব্যবহার করা যাবে না।

- ভাষার ধরন সীমিত।** ট্রেনিং ডেটা ইংরেজি টুইট, যার অধিকাংশ "i feel..." দিয়ে শুরু। আনুষ্ঠানিক লেখা, দীর্ঘ নথি বা অন্য ভাষায় ফলাফল অনির্ভরযোগ্য।
- ছয়টি শ্রেণি, সবসময় একজন বিজয়ী।** নিরপেক্ষ, মিশ্র বা ব্যঙ্গাত্মক বাক্যের কোনো সঠিক উত্তর নেই — তবু মডেলকে একটি বাছতেই হয়। তাই কম আত্মবিশ্বাসের ফলাফলে দ্বিতীয় স্থানের স্কোর দেখা জরুরি।
- বিরল শ্রেণি দুর্বল।** `surprise` -এর recall মাত্র ০.৬৮।
- পক্ষপাত উত্তরাধিকারসূত্রে পাওয়া।** মূল ডেটা ও DistilBERT-এর প্রি-ট্রেনিং ডেটায় থাকা সামাজিক পক্ষপাত এই মডেলেও রয়ে গেছে; ফাইন-টিউনিং তা সংশোধন করে না।
- ডেটাসেট লাইসেন্স।** গবেষণা ও শিক্ষামূলক ব্যবহারের জন্য — বাণিজ্যিক ব্যবহারে পৃথক অনুমতি প্রয়োজন।

১৭. সুপারভাইজারের সম্ভাব্য প্রশ্ন ও উত্তর

ক. সততা ও পরিধি সংক্রান্ত

প্রশ্ন: তুমি নিজে মডেল ট্রেন করেছ?

বেসলাইন মডেল (TF-IDF + Logistic Regression) আমি নিজে ট্রেন করেছি — টেস্টে ৮৭.১৫% accuracy। ট্রান্সফরমার ফাইন-টিউনিং-এর সম্পূর্ণ কোড ও Kaggle নোটবুক আমি লিখেছি, কিন্তু **ফাইন-টিউনিং এখনো চালানো হয়নি**। বর্তমানে একই ডেটাসেটে ট্রেন করা একটি পাবলিক DistilBERT ব্যবহার করছি এবং **সেটি আমি নিজে টেস্ট সেটে মূল্যায়ন করেছি** — ৯২.৭০% accuracy, ০.৮৮২৫ macro-F1।

প্রশ্ন: তাহলে তোমার নিজের অবদান কী?

সম্পূর্ণ ML পাইপলাইন (ডেটা বিশ্লেষণ, বেসলাইন ট্রেনিং, ফাইন-টিউনিং কোড, মূল্যায়ন কোড), সম্পূর্ণ ব্যাকএন্ড API, সম্পূর্ণ ফ্রন্টএন্ড, ৫৮টি টেস্ট, Docker কনটেইনারাইজেশন, CI পাইপলাইন এবং লাইভ ডিপ্লয়মেন্ট। মডেল ওজন ছাড়া বাকি সবটাই।

প্রশ্ন: নিজের মডেল ট্রেন করতে কত সময় লাগবে?

Kaggle-এর ফ্রি T4 GPU-তে প্রায় ৬ মিনিট। নোটবুকটি প্রস্তুত — শুধু GPU চালু করে, HF টোকেন যোগ করে চালালেই হবে। এরপর ব্যাকএন্ডে `MODEL_REPO` বদলালেই লাইভ সাইট নিজের মডেল ব্যবহার শুরু করবে।

খ. ট্রেনিং সংক্রান্ত

প্রশ্ন: Fine-tuning মানে কী? শূন্য থেকে ট্রেন করলে না কেন?

শূন্য থেকে ট্রেন করতে কোটি টাকার GPU ও সপ্তাহের পর সপ্তাহ লাগত। Fine-tuning-এ গুগলের আগে থেকে ট্রেন করা DistilBERT নেওয়া হয় — যেটি ইতিমধ্যে ইংরেজি ভাষা জানে — এবং শুধু আমাদের নির্দিষ্ট কাজের জন্য সামান্য সমন্বয় করা হয়। এটি Transfer Learning-এর প্রয়োগ।

প্রশ্ন: প্রি-ট্রেনিং-এ মডেল কী শেখে?

Masked Language Modeling — বাক্য থেকে এলোমেলোভাবে শব্দ লুকিয়ে দিয়ে সেটি অনুমান করতে বলা হয়। কোটি কোটি বাক্যে এটি করতে করতে মডেল ব্যাকরণ, শব্দের অর্থ ও প্রেক্ষাপট শিখে ফেলে। এতে কোনো মানুষের লেবেল লাগে না — তাই একে self-supervised learning বলে।

প্রশ্ন: Learning rate 2e-5 কেন? বেশি দিলে কী হতো?

প্রি-ট্রেন করা ওজনগুলো ইতিমধ্যে ভালো অবস্থায় আছে; বড় learning rate দিলে সেই মূল্যবান জ্ঞান নষ্ট হয়ে যেত — একে বলে catastrophic forgetting। আবার খুব ছোট দিলে শেখা অসম্ভব ধীর হতো। ফাইন-টিউনিং-এ 2e-5 থেকে 5e-5 হলো প্রমাণিত পরিসর।

প্রশ্ন: Warmup কেন দরকার?

ট্রেনিং শুরুতে classification head সম্পূর্ণ এলোমেলো, তাই তখনকার gradient খুব বিশৃঙ্খল। ওই সময় পূর্ণ learning rate দিয়ে বড় পদক্ষেপ নিলে প্রি-ট্রেন করা ভালো ওজনগুলো ক্ষতিগ্রস্ত হতে পারে। তাই প্রথম ৬% ধাপে learning rate শূন্য থেকে ধীরে ধীরে বাড়ানো হয়।

প্রশ্ন: Backpropagation কীভাবে কাজ করে?

Forward pass-এ ভবিষ্যদ্বাণী তৈরি হয়, তারপর loss হিসাব হয়। Backpropagation ক্যালকুলাসের চেইন রুল ব্যবহার করে হিসাব করে প্রতিটি প্যারামিটার সেই ভুলের জন্য কতটা দায়ী — এটিই gradient। এরপর optimizer প্রতিটি ওজনকে ভুল কমানোর দিকে সামান্য সরিয়ে দেয়।

প্রশ্ন: Epoch কয়টা এবং মোট কতগুলো আপডেট হয়?

৫টি epoch। ১৬,০০০ বাক্য ÷ ৩২ ব্যাচ = ৫০০ step প্রতি epoch, অর্থাৎ মোট ২,৫০০ বার ওজন আপডেট হয়। প্রতিটি বাক্য মডেল ৫ বার দেখে।

প্রশ্ন: Overfitting কীভাবে ঠেকিয়েছ?

চারভাবে — (১) প্রতি epoch-এ ভ্যালিডেশন সেটে যাচাই, (২) সর্বোচ্চ ভ্যালিডেশন macro-F1 পাওয়া চেকপয়েন্ট নির্বাচন (শেষেরটি নয়), (৩) weight decay 0.01, (৪) মাত্র ৫টি epoch।

প্রশ্ন: Loss function কী ব্যবহার করেছ?

Cross-Entropy Loss, সূত্র $-\log(P_{\text{সঠিক শ্রেণি}})$ । এটি আত্মবিশ্বাসের সাথে ভুল করাকে সবচেয়ে কঠোরভাবে শাস্তি দেয় — যা কাম্য, কারণ আমরা চাই মডেল নিশ্চিত না হলে দ্বিধা প্রকাশ করুক।

গ. ডেটা সংক্রান্ত

প্রশ্ন: টেস্ট সেট আলাদা রাখা কেন জরুরি?

মডেল যদি পরীক্ষার প্রশ্ন আগে দেখে ফেলে, তার নম্বর অর্থহীন হয়ে যায় — সেটি প্রকৃত শেখার প্রমাণ দেয় না। আমাদের ২,০০০টি টেস্ট বাক্য ট্রেনিং বা চেকপয়েন্ট নির্বাচন — কোনোটিতেই ব্যবহৃত হয়নি। তাই ৯২.৭০% সংখ্যাটি বিশ্বাসযোগ্য।

প্রশ্ন: Validation আর Test আলাদা কেন? একটা হলেই তো হতো?

না। ভ্যালিডেশন সেট দেখে আমরা সিদ্ধান্ত নিই (কোন চেকপয়েন্ট রাখব, কোন হাইপারপ্যারামিটার ভালো)। বারবার দেখে সিদ্ধান্ত নেওয়ার ফলে মডেল পরোক্ষভাবে ওই সেটের সাথে খাপ খেয়ে যায়। তাই সম্পূর্ণ নিরপেক্ষ চূড়ান্ত রায়ের জন্য তৃতীয় একটি সেট দরকার, যা কখনোই দেখা হয়নি।

প্রশ্ন: ডেটা ভারসাম্যহীন — এতে কী সমস্যা?

joy ৩৩.৫১% আর surprise মাত্র ৩.৫৮% — ৯.৪ গুণ পার্থক্য। ফলে একটি মডেল surprise সম্পূর্ণ উপেক্ষা করেও ভালো accuracy পেতে পারে। আমাদের ফলাফলেই এর প্রমাণ আছে: surprise-এর recall মাত্র ০.৬৮, অথচ sadness-এর ০.৯৭।

প্রশ্ন: Stopword সরানো কেন?

ইচ্ছাকৃতভাবে। "not", "i", "am" — এই শব্দগুলোই আবেগের জন্য অত্যন্ত গুরুত্বপূর্ণ। "I am not happy" থেকে "not" সরালে অর্থ সম্পূর্ণ উল্টে যায়। ঐতিহ্যগত টেক্সট প্রসেসিং-এ stopwords সরানো হয়, কিন্তু আবেগ শনাক্তকরণে সেটি ক্ষতিকর।

ঘ · মূল্যায়ন সংক্রান্ত

প্রশ্ন: Accuracy না দেখে macro-F1 দেখছে কেন?

ভারসাম্যহীন ডেটায় accuracy বিভ্রান্তিকর। Macro-F1 ছয়টি শ্রেণির F1-এর সরল গড় নেয়, তাই ৬৬ নমুনার surprise আর ৬৯৫ নমুনার joy সমান গুরুত্ব পায়। আমাদের ফলাফলেই পার্থক্যটা স্পষ্ট — macro-F1 ০.৮৮২৫ কিন্তু weighted-F1 ০.৯২৬৯, কারণ পরেরটি বিরল শ্রেণির দুর্বলতা ঢেকে দেয়।

প্রশ্ন: Precision আর Recall-এর পার্থক্য বলো।

Precision = মডেল যতবার "joy" বলেছে তার কত ভাগ সঠিক ছিল। Recall = ডেটায় যত আসল joy ছিল তার কত ভাগ মডেল ধরতে পেরেছে। Precision ভুল অ্যালার্ম মাপে, Recall মিস করা মাপে। F1 হলো দুটোর হারমোনিক গড়।

প্রশ্ন: হারমোনিক গড় কেন, সাধারণ গড় নয়?

কারণ হারমোনিক গড়ে যেকোনো একটি মান খুব কম হলেই ফলাফল কমে যায়। যেমন precision ১.০০ আর recall ০.০২ হলে সাধারণ গড় ০.৫১ (ভালো দেখায়) কিন্তু F1 মাত্র ০.০৪ — যা বাস্তবতার সঠিক প্রতিফলন, কারণ মডেলটি কার্যত অকেজো।

প্রশ্ন: Confusion matrix থেকে কী শিখলে?

তিনটি প্রধান ভুল — (১) joy ↔ love মোট ৫৫টি, (২) surprise → fear ১৫টি, (৩) anger ↔ sadness ২০টি। গুরুত্বপূর্ণ হলো এগুলো এলোমেলো ভুল নয়, **ভাষাগতভাবে যুক্তিসঙ্গত** — joy আর love প্রকৃতিগতভাবেই ঘনিষ্ঠ, আর বিস্ময় প্রায়ই ভয়ের ভাষা ব্যবহার করে।

প্রশ্ন: বেসলাইন কেন বানাতে?

তুলনার মানদণ্ড ছাড়া "৯২.৭% ভালো" কথাটির কোনো অর্থ নেই। বেসলাইন দেখাল একটি সরল পদ্ধতি ৮৭.১৫% পায়, তাই ট্রান্সফরমারের +৫.৫৫ পয়েন্ট উন্নতি এর জটিলতা ও খরচকে ন্যায্যতা দেয়। এটি ছাড়া দাবিটি প্রমাণ করা যেত না।

ঙ . স্থাপত্য ও প্রকৌশল সংক্রান্ত

প্রশ্ন: BERT-এর বদলে DistilBERT কেন?

ফ্রি হোস্টিং-এ RAM মাত্র ৫১২ MB, GPU নেই। DistilBERT ৪০% ছোট ও ৬০% দ্রুত, অথচ BERT-এর ~৯৭% কর্মদক্ষতা ধরে রাখে। এটি একটি সচেতন প্রকৌশল সমঝোতা — সামান্য নির্ভুলতার বিনিময়ে বাস্তবে চালানোর সক্ষমতা।

প্রশ্ন: Attention কী এবং কেন গুরুত্বপূর্ণ?

Attention মডেলকে সিদ্ধান্ত নিতে দেয় একটি শব্দ প্রক্রিয়াকরণের সময় বাক্যের কোন শব্দগুলো বেশি গুরুত্বপূর্ণ। "I am not happy" বাক্যে "happy" দেখার সময় মডেল "not"-এ বেশি মনোযোগ দেয় এবং বোঝে অর্থ নেতিবাচক। TF-IDF শুধু শব্দ গণনা, এই সম্পর্কটি ধরতে পারে না — এখানেই ট্রান্সফরমারের মূল শ্রেষ্ঠত্ব।

প্রশ্ন: [CLS] টোকেন কী কাজ করে?

এটি বাক্যের শুরুতে বসে একটি বিশেষ টোকেন। সব স্তর পার হওয়ার পর এর ভেক্টরটি পুরো বাক্যের সারসংক্ষেপ ধারণ করে, এবং শ্রেণিবিন্যাসের সিদ্ধান্ত এই একটি ৭৬৮-মাত্রার ভেক্টর থেকেই নেওয়া হয়।

প্রশ্ন: মডেল লোড হতে ৩০ সেকেন্ড লাগলে ব্যবহারকারী কী দেখেন?

মডেল একটি ব্যাকগ্রাউন্ড থ্রেডে লোড হয়, তাই `/api/health` সাথে সাথে `status: "loading"` ফেরত দিতে পারে। ফ্রন্টএন্ড সেটি দেখে "মডেল জেগে উঠছে, এক মিনিট পর্যন্ত লাগতে পারে" বার্তা দেখায় — সাদা পর্দা বা ভাঙা অ্যাপ নয়।

প্রশ্ন: মডেল লোড না হলে কী হয়?

সিস্টেম বন্ধ হয় না — বেসলাইন মডেলে নেমে আসে এবং প্রতিটি উত্তরে `model_source: "baseline"` লিখে সৎভাবে জানায় যে এটি দুর্বল মডেলের ফলাফল। একে বলে graceful degradation।

প্রশ্ন: প্রতি অক্ষরে API ডাকো না কেন?

Debouncing ব্যবহার করেছি — টাইপ থামার ৪০০ms পরে একবার অনুরোধ যায়। এছাড়া নতুন অনুরোধ শুরু হলে আগেরটি বাতিল করা হয়, যাতে পুরনো ধীর উত্তর নতুন সঠিক উত্তরকে চাপা না দেয়।

প্রশ্ন: ব্যবহারকারীর লেখা কি সংরক্ষণ করো?

না। লগে শুধু টেক্সটের দৈর্ঘ্য, ফলাফলের লেবেল ও সময় রাখা হয় — লেখার বিষয়বস্তু কোথাও সংরক্ষিত হয় না। সেশন ইতিহাসও শুধু ব্যবহারকারীর নিজের ব্রাউজারে থাকে।

চ. সীমাবদ্ধতা সংক্রান্ত

প্রশ্ন: এই সিস্টেমের সবচেয়ে বড় দুর্বলতা কী?

`surprise` শ্রেণির recall মাত্র ০.৬৮১৮ — ৬৬টির মধ্যে ২১টি মিস করে, যার ১৫টি ভুল করে fear বলে। কারণ ট্রেনিং ডেটায় surprise মাত্র ৩.৫৮%। এটি অ্যাপের About পাতায় স্পষ্টভাবে দেখানো আছে, লুকানো হয়নি।

প্রশ্ন: বাংলা বা অন্য ভাষায় কাজ করবে?

না। মডেলটি শুধু ইংরেজিতে ট্রেন করা। অন্য ভাষা দিলে উত্তর আসবে, কিন্তু তা অর্থহীন হবে। বহুভাষিক সমর্থনের জন্য XLM-RoBERTa-র মতো মডেল লাগত।

প্রশ্ন: ব্যঙ্গ (sarcasm) ধরতে পারবে?

না। "বাহ, চমৎকার দিন" — ব্যঙ্গ হলে এটি রাগ বোঝায়, কিন্তু শব্দগুলো আনন্দের। ব্যঙ্গ বুঝতে সুর, প্রেক্ষাপট ও বক্তার সম্পর্কে জ্ঞান লাগে, যা শুধু লেখা থেকে পাওয়া যায় না।

প্রশ্ন: এটি মানসিক স্বাস্থ্য যাচাইয়ে ব্যবহার করা যাবে?

একেবারেই না। এটি চিকিৎসা বা রোগনির্ণয়ের যন্ত্র নয় — শুধু শব্দ দেখে অনুমান করে। মানসিক স্বাস্থ্য, নিয়োগ, বা কোনো ব্যক্তির অধিকার সংক্রান্ত সিদ্ধান্তে ব্যবহার করা নৈতিকভাবে অগ্রহণযোগ্য। এই সতর্কবার্তা অ্যাপের ফুটার ও About পাতায় লেখা আছে।

প্রশ্ন: ফলাফল আরও ভালো করতে কী করতে?

(১) surprise-এর জন্য class weights চালু করা বা oversampling, (২) DistilBERT-এর বদলে RoBERTa বা DeBERTa ব্যবহার, (৩) একাধিক মডেলের ensemble, (৪) joy/love-এর মতো ঘনিষ্ঠ শ্রেণির জন্য বাড়তি ডেটা সংগ্রহ, (৫) হাইপারপ্যারামিটার অনুসন্ধান।

১৮. পরিভাষা — ইংরেজি ও বাংলা

ইংরেজি	বাংলা	অর্থ
Machine Learning	মেশিন লার্নিং	ডেটা থেকে নিয়ম শেখার পদ্ধতি
Supervised Learning	তত্ত্বাবধানে শেখা	সঠিক উত্তর সহ শেখানো
Classification	শ্রেণিবিন্যাস	নির্দিষ্ট কয়েকটি ভাগে ফেলা
Label	লেবেল	একটি উদাহরণের সঠিক উত্তর
Feature	বৈশিষ্ট্য	ইনপুটের যে তথ্য থেকে সিদ্ধান্ত নেওয়া হয়
Train / Validation / Test	প্রশিক্ষণ / যাচাই / পরীক্ষা	ডেটার তিনটি ভাগ
Overfitting	অতি-অভিযোজন	বোঝার বদলে মুখস্থ করা
Regularization	নিয়মিতকরণ	ওভারফিটিং ঠেকানোর কৌশল
Tokenization	টোকেনাইজেশন	টেক্সটকে টুকরায় ভাগ করা
Embedding	এমবেডিং	শব্দের অর্থ বহনকারী ভেক্টর
Transformer	ট্রান্সফরমার	attention-ভিত্তিক নিউরাল স্থাপত্য
Attention	মনোযোগ	কোন শব্দ বেশি গুরুত্বপূর্ণ তা নির্ধারণ
Pre-training	প্রাক-প্রশিক্ষণ	বিপুল সাধারণ ডেটায় প্রাথমিক শেখা
Fine-tuning	সূক্ষ্ম সমন্বয়	নির্দিষ্ট কাজের জন্য অল্প সমন্বয়
Transfer Learning	জ্ঞান স্থানান্তর	এক কাজের শেখা অন্য কাজে লাগানো
Epoch	ইপক	পুরো ডেটা একবার দেখা
Batch	ব্যাচ	একসাথে প্রক্রিয়াকৃত নমুনার দল
Learning Rate	শিখন হার	প্রতি ধাপে ওজন কতটা বদলাবে
Loss Function	ক্ষতি ফাংশন	ভুলের পরিমাপ
Gradient	ঢাল	ভুল কমাতে কোন দিকে যেতে হবে
Backpropagation	পশ্চাৎ-প্রচারণ	দায় নির্ণয় করে ঢাল হিসাব
Optimizer	অপটিমাইজার	ওজন হালনাগাদের অ্যালগরিদম
Softmax	সফটম্যাক্স	স্কেরকে সম্ভাব্যতায় রূপান্তর
Logits	লজিট	softmax-এর আগের কাঁচা স্কের
Inference	ইনফারেন্স	ট্রেন করা মডেল দিয়ে ভবিষ্যদ্বাণী
Checkpoint	চেকপয়েন্ট	মডেলের সংরক্ষিত অবস্থা
Accuracy	নির্ভুলতা	মোট সঠিক উত্তরের হার
Precision	শুদ্ধতা	বলা উত্তরগুলোর কত ভাগ সঠিক

ইংরেজি	বাংলা	অর্থ
Recall	পুনরুদ্ধার	আসল উদাহরণের কত ভাগ ধরা পড়ল
F1-Score	এফ-১ স্কোর	precision ও recall-এর হারমোনিক গড়
Macro-F1	ম্যাক্রো এফ-১	সব শ্রেণির F1-এর সরল গড়
Confusion Matrix	বিভ্রান্তি ছক	কোন শ্রেণি কোনটির সাথে গুলিয়েছে
Class Imbalance	শ্রেণি ভারসাম্যহীনতা	শ্রেণিগুলোর অসম পরিমাণ
Baseline	বেসলাইন	তুলনার জন্য সরল মডেল
API	এপিআই	প্রোগ্রামের মধ্যে যোগাযোগের মাধ্যম
Endpoint	এন্ডপয়েন্ট	API-র একটি নির্দিষ্ট ঠিকানা
Docker	ডকার	পরিবেশসহ অ্যাপ প্যাকেজ করার প্রযুক্তি
Cold Start	ঠান্ডা শুরু	ঘুমন্ত সার্ভার জাগার বিলম্ব
Debouncing	ডিবাউন্সিং	থামার পর একবার অনুরোধ পাঠানো
Graceful Degradation	সহনীয় অবনমন	সম্পূর্ণ বন্ধ না হয়ে সীমিত সেবা

শেষ কথা — উপস্থাপনার সময় মনে রাখুন

1. **সং থাকুন।** ট্রান্সফরমারটি আপনার নিজের ট্রেন করা নয় — এটি স্পষ্টভাবে বলুন। সততা দুর্বলতা নয়, এটি পেশাদারিত্ব।
2. **বেসলাইনের কথা বলুন।** এটি আপনার নিজের ট্রেন করা এবং এটি প্রমাণ করে আপনি তুলনামূলক পদ্ধতি বোঝেন।
3. **macro-F1-এর যুক্তি বলুন।** এটি দেখায় আপনি শুধু বড় সংখ্যা নয়, সঠিক সংখ্যা খুঁজছেন।
4. **surprise-এর দুর্বলতা নিজে থেকে বলুন।** সীমাবদ্ধতা স্বীকার করা গবেষকের গুণ।
5. **confusion matrix-এর ব্যাখ্যা দিন।** ভুলগুলো যুক্তিসঙ্গত — এটি বোঝা মানে গভীরে যাওয়া।